CS3423 Compilers II
Mini-Assignment 4: Clang AST Passes

CS24BTECH11030, CS24BTECH11036

October 2026

1 Part 1: Exploring AST dumps

1.1 Program

The input is `nested_loops.cpp`: a function `f` containing a loop, and a `main` with a doubly nested loop (both loops use a variable called `i`) and a call to `f`.

```
#include <stdio.h>

void f(int n){
    for(int i = 0; i < n; i++){
        printf("Hello World\n");
    }
}

int main(){
    int n = 10;
    for(int i = 0; i < n; i++){
        for(int i = 0; i < n; i++){
            printf("Hello World\n");
        }
    }
    f(n);
    return 0;
}
```

1.2 Dumping the AST

```
clang -fsyntax-only -Xclang -ast-dump -Xclang -ast-dump-filter=main nested_loops.cpp
```

`-ast-dump-filter=main` restricts the dump to the declaration named `main`; without it the headers pulled in by `<stdio.h>` dominate the output. The dump below has addresses and most source ranges removed for readability (the raw output is `ast_dump_main.txt`).

```
FunctionDecl main 'int ()'
`-CompoundStmt
  |-DeclStmt
  | `-VarDecl used n 'int' cinit
  |   `-IntegerLiteral 'int' 10
  |-ForStmt                                    <-- outer loop
  | |-DeclStmt
  | | `-VarDecl used i 'int' cinit
  | |   `-IntegerLiteral 'int' 0
```

```
| |-<<<NULL>>>                              <-- no condition variable
| |-BinaryOperator 'bool' '<'
| | |-ImplicitCastExpr 'int' <LValueToRValue>
| | | `-DeclRefExpr 'int' lvalue Var 'i'
| | `-ImplicitCastExpr 'int' <LValueToRValue>
| |   `-DeclRefExpr 'int' lvalue Var 'n'
| |-UnaryOperator 'int' postfix '++'
| | `-DeclRefExpr 'int' lvalue Var 'i'
| `-CompoundStmt
|   `-ForStmt                               <-- inner loop
|     |-DeclStmt ... (VarDecl i)
|     |-<<<NULL>>>
|     |-BinaryOperator 'bool' '<'
|     |-UnaryOperator 'int' postfix '++'
|     `-CompoundStmt
|       `-CallExpr 'int'                     <-- printf(...)
|         |-ImplicitCastExpr <FunctionToPointerDecay>
|         | `-DeclRefExpr Function 'printf'
|         `-ImplicitCastExpr <ArrayToPointerDecay>
|           `-StringLiteral "Hello World\n"
|-CallExpr 'void'                            <-- f(n)
| |-ImplicitCastExpr <FunctionToPointerDecay>
| | `-DeclRefExpr Function 'f'
| `-ImplicitCastExpr <LValueToRValue>
|   `-DeclRefExpr Var 'n'
`-ReturnStmt
  `-IntegerLiteral 'int' 0
```

1.3 Interesting nodes and their hierarchy

1. `ForStmt`. A `for` loop is one node with four fixed child slots, in order: *init* (`DeclStmt`), *condition variable* (here `<<<NULL>>>`, used only for `for (...; T v = ...; )` forms), *condition* (`BinaryOperator <`), *increment* (`UnaryOperator ++`), and finally the *body*. In the dump the body is a `CompoundStmt`, whose only child is the inner `ForStmt`. This is how loop nesting appears in the tree: the inner loop is a child of the outer loop's body, not a sibling.

2. `CallExpr`. `printf("Hello World\n")` is a `CallExpr` whose first child is the callee expression and the remaining children are the arguments. The callee is not a bare `DeclRefExpr`: Clang wraps it in an `ImplicitCastExpr <FunctionToPointerDecay>`, so the node under `CallExpr` is the cast, and the `DeclRefExpr` naming `printf` is its child. The string argument is likewise wrapped in an `ArrayToPointerDecay` cast above the `StringLiteral`. A matcher for "calls to `printf`" therefore has to look through these implicit nodes, which `callee(functionDecl(hasName("printf")))` does for us.

3. `DeclStmt` / `VarDecl` / `DeclRefExpr`. `int i = 0;` is a `DeclStmt` (a statement) holding a `VarDecl` (the declaration), whose child is the initializer `IntegerLiteral`. Every later use of `i` is a `DeclRefExpr`, which does not copy the variable but *refers* to a particular `VarDecl` (the dump shows `Var 0x... 'i'` before the addresses were stripped). The program declares `i` twice, so there are two distinct `VarDecl` nodes; each `DeclRefExpr` points to the nearest enclosing one, so the inner loop's `i` shadows the outer one. This declaration/reference link is exactly what is needed to decide whether a local variable is ever used (Part 2, pass 3).

Other nodes worth noting. `ImplicitCastExpr <LValueToRValue>` appears around every variable that is *read* (`i` and `n` in `i < n`) but not around `i` in `i++`, where it is modified in place. `CompoundStmt` is the `{ ... }` block and just holds a list of statements. The overall root of this dump is `FunctionDecl`, which sits under the `TranslationUnitDecl`.

2 Traversal mechanisms in Clang

`RecursiveASTVisitor`. A CRTP base class. Derive from `RecursiveASTVisitor<Derived>` and define `VisitXxx(Xxx *node)` for the node kinds of interest; the base class walks the whole tree depth-first and calls them. Three layers exist per node kind: `TraverseXxx` (controls descent into children and can prune a subtree by not calling the base), `WalkUpFromXxx` (calls the `Visit` methods for the node and its base classes), and `VisitXxx` (what you usually override; return `false` to stop the whole traversal). Best for imperative analyses that need state across the walk.

AST matchers with `MatchFinder`. A declarative approach: describe the pattern as a composition of matchers (node matchers such as `forStmt`, `varDecl`; narrowing matchers such as `hasName`; traversal matchers such as `hasBody`, `hasCondition`, `hasDescendant`) and attach a `MatchCallback` to it with `MatchFinder::addMatcher`. `.bind("id")` labels sub-nodes, which the callback's `run` retrieves with `Result.Nodes.getNodeAs<T>("id")`. The finder does the walk internally. Concise for local patterns, and what Part 2 uses.

`ASTConsumer` and `ASTFrontendAction`. The hook by which a tool gets the AST at all. A `FrontendAction` creates an `ASTConsumer`, whose `HandleTranslationUnit(ASTContext&)` (or `HandleTopLevelDecl`) is called once parsing finishes. From there one typically starts a `RecursiveASTVisitor` from `Context.getTranslationUnitDecl()`. `newFrontendActionFactory(&Finder)` in a LibTooling `main` hides this for matchers.

`StmtVisitor` / `DeclVisitor`. Dispatch-only visitors: `Visit(node)` calls the `VisitXxx` for that node's dynamic type, but *does not* recurse; you recurse manually via `node->children()`. Used inside Clang itself (e.g. the AST printer, code generation) when the traversal order is custom.

Direct child iteration and parent queries. Every `Stmt` exposes `children()`, and a `DeclContext` exposes `decls()`, for a hand-written walk. Clang nodes have no parent pointer; `ASTContext::getParents(node)` (backed by the `ParentMapContext`) computes the parent map on demand for upward navigation.

3 Part 2: AST-matcher passes

All three checks are in a single LibTooling tool built from `pass.cpp` (target `Simple_Pass`), run as `Simple_Pass file.cpp --`. Each check has a test input in `tests/`.

3.1 Build and run

```
cp -r 2-ast-matcher-passes ~/llvm-project/clang-tools-extra/Mypass
# add:  add_subdirectory(Mypass)  to clang-tools-extra/CMakeLists.txt
cd ~/llvm-project/build && cmake . && make -j4 Simple_Pass
bin/Simple_Pass ../clang-tools-extra/Mypass/tests/assign_in_condition.cpp --
```

Tested against LLVM/Clang 17.0.6.

3.2 Pass 1: assignment inside a condition

Goal. Flag `if (x = 5)`, where `==` was probably intended.
Approach. The five statement kinds that have a condition (`if`, `for`, `while`, `do-while` and the `?:`

operator) each get a matcher of the form shown below (here for `if`), and the five are combined with `anyOf` under one `stmt(..., isExpansionInMainFile())`.

```
ifStmt(hasCondition(ignoringParenImpCasts(
    binaryOperator(hasOperatorName("=")).bind("assignment"))))
```

`ignoringParenImpCasts` is needed because in C++ the condition `n = 13` is wrapped in an `ImplicitCastExpr` (int to bool), and `if ((n = 13))` adds a `ParenExpr`; both forms are therefore found. Only plain `=` is matched; compound assignments such as `+=` are not reported. The bind ID is placed on the `binaryOperator`, so the callback receives the assignment itself and prints its source location.
Test. `tests/assign_in_condition.cpp`
Output. Only the assignment on line 5 is reported; the comparison `n == 10` on line 8 is not.

```
tests/assign_in_condition.cpp:5:8
```

3.3 Pass 2: empty loop body

Goal. Flag `for`, `while` and `do-while` loops whose body is an empty compound statement {}.
Approach. `forStmt`, `whileStmt` and `doStmt` each use `hasBody(compoundStmt(statementCountIs(0)))`, i.e. the body must be a block containing no statements, and all three bind the same ID `loop`. A loop whose body is a single statement or a bare `;` (`NullStmt`, as in `for (...);`) is deliberately not reported, since the question asks for empty compound statements. The matchers are restricted to the main file.
Test. `tests/empty_loop_body.cpp`
Output. Only the empty `for` on line 6 is reported; the `do` and `while` loops in the test have non-empty bodies.

```
tests/empty_loop_body.cpp:6:5
```

3.4 Pass 3: unused local variable

Goal. Flag local variables that are declared but never referenced in their enclosing function.
Approach. Two matchers feed one callback: `varDecl(hasLocalStorage(), unless(parmVarDecl()))` records every local variable declaration, and `declRefExpr(to(varDecl(hasLocalStorage())))` records every use. Each variable has exactly one `VarDecl` node, and each use is a `DeclRefExpr` whose `getDecl()` points back to it, so the `VarDecl*` identifies the variable (two variables both called `i` are different pointers). The callback keeps a set of declared and a set of referenced pointers; after `Tool.run` the variables in the first set but not in the second are printed. The report cannot be produced inside the callback because a use may appear after the declaration. Parameters, globals and `static` locals are not reported. Any reference counts as a use, so a variable that is only written (`int c = 3; c = 4;`) is not reported.
Test. `tests/unused_local.cpp`
Output. The unused `x` in `f` and the unused `x` in `main` are both reported; `n` and `i` are used and are not.

```
Unused variable: x
Unused variable: x
```

4 Limitations

- Pass 1 reports only plain `=`; `if (x += 2)` is not reported. An assignment hidden in a function call or a lambda inside the condition is not found, because the matcher inspects only the condition expression itself.
- Pass 2 does not report a `for (...);` or `while (...);` loop (a `NullStmt` body) or a body containing only an empty nested block.
- Pass 3 counts any reference as a use, so write-only variables are missed; it does not report unused parameters, globals or `static` locals, and the report prints only the variable name (no source location).
- All passes ignore code that comes from headers (`isExpansionInMainFile`), so problems inside included files are not reported. Macros and templates were not tested.

Note on AI use

The text of this write-up was drafted with AI assistance (Claude). The passes themselves were written by the authors.